

Web3 AI Smart Router & Security Infrastructure

An Enterprise-Grade, Fault-Tolerant, and Latency-Optimized Web3 RPC Proxy & Control Center Powered by Google Gemini AI

AUTHOR & LEAD ENGINEER
Adityaveer Singh Chauhan

ACADEMIC AFFILIATION
B.Tech Civil Engineering (2nd Year), IIT Roorkee

ENROLLMENT NUMBER
25113013

REPOSITORY ARCHITECTURE
Express Middleware / Gemini AI RPC Proxy

1. Problem Statement & Systemic Web3 Vulnerabilities

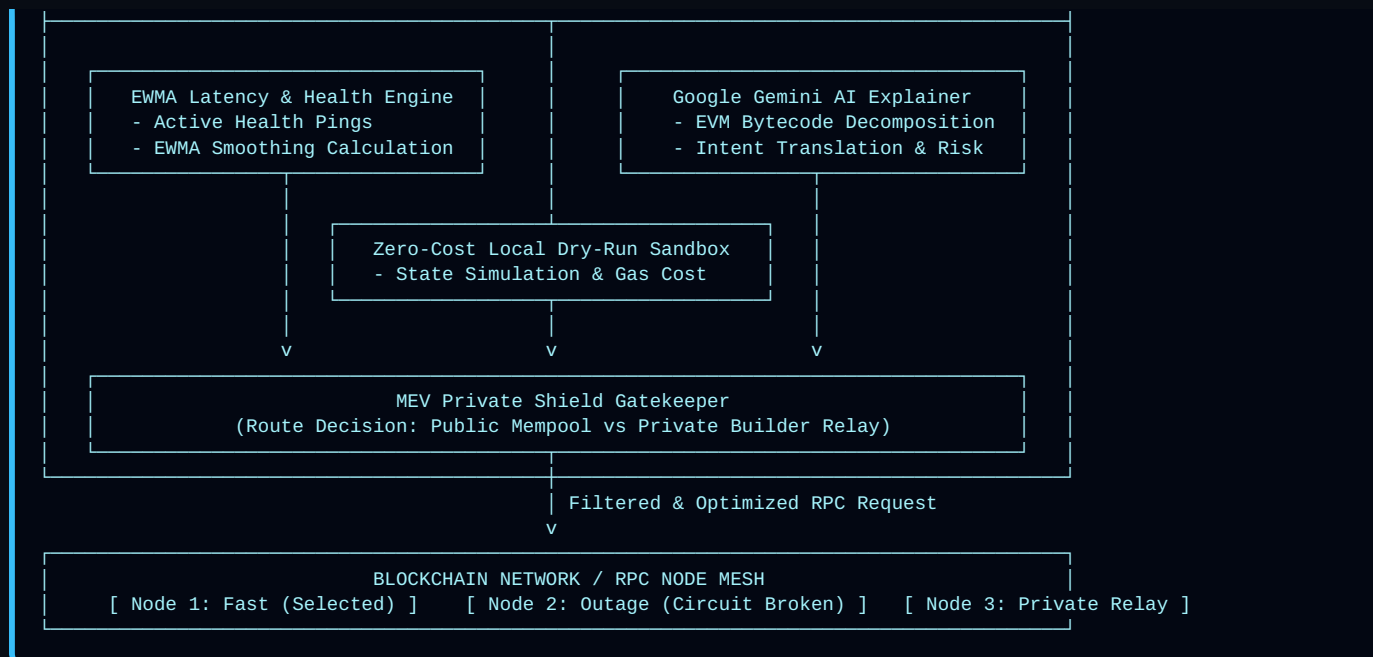
The vast majority of modern Web3 decentralization is strictly confined to on-chain smart contract logic. At the application edge, user interactions rely heavily on centralized Remote Procedure Call (RPC) nodes provided by static third-party endpoints. This design pattern introduces critical vulnerabilities that compromise security, reliability, and censorship resistance:

Vulnerability Type	Root Technical Cause	Impact on Web3 End Users & dApps
1. Single Point of Failure (SPOF) Outages	Wallets hardcode static RPC URLs. If the provider experiences downtime, cloud rate-limiting, or BGP routing failures, the dApp becomes unresponsive.	Complete transaction blackout, failed dApp connections, and inability to manage positions during market volatility.
2. Unreadable Calldata & Drainer Phishing	Ethereum Virtual Machine (EVM) transactions are signed as raw hexadecimal byte payloads (<code>0xa9059cbb...</code>). Wallets fail to decode complex proxy interactions.	Users blind-sign malicious approvals (<code>permit</code> / <code>setApprovalForAll</code>), leading to wallet drainer exploits and asset theft.
3. Public Mempool Exposure (MEV Exploits)	Pending transaction write requests are broadcast to public mempools, exposing parameters to automated arbitrage and sandwich bots.	Extreme slippage, front-running attacks, and financial losses due to Maximal Extractable Value (MEV) bot exploitation.
4. RPC Endpoint Level Censorship	Centralized RPC nodes log source IP addresses and selectively filter or drop transactions originating from specific regions or interacting with blacklisted contracts.	Direct violation of Web3 censorship resistance, enabling centralized entities to unilaterally block user access to public blockchains.

2. System Architecture & Component Interaction

The **Web3 AI Smart Router** functions as a local or enterprise-hosted reverse proxy positioned between the client's wallet layer (e.g., MetaMask, Rabby) and a heterogeneous mesh of external RPC nodes. Incoming JSON-RPC calls are intercepted, analyzed, enriched, and routed based on live mathematical telemetry and AI intent extraction.





3. Alignment with Devcon Track: Censorship Resistance

In decentralized infrastructure, censorship resistance must exist at the transport and RPC access layers—not just within block validation. The Web3 AI Smart Router establishes three core pillars of client-side anti-censorship:

PILLARS OF CLIENT-SIDE ANTI-CENSORSHIP

- **Autonomous Multi-Node Mesh Routing:** Eliminates reliance on any single provider (e.g., Infura, Alchemy). If a primary provider censors a transaction or restricts an IP address, the EWMA engine detects anomalous drops and immediately reroutes traffic to censorship-free alternative endpoints.
- **Mempool Bypass via Private Relay Inclusion:** By integrating direct pathways to private block builders (Flashbots, MEV-Share), transactions bypass public peer-to-peer mempools. This prevents malicious node operators and mempool listeners from selectively dropping or front-running user transactions.
- **Immutable Client-Side Sovereignty:** Operating as a user-controlled local proxy server ensures that transaction filtering rules, AI prompts, and security checks are executed entirely on the user's infrastructure, free from third-party administrative override.

4. Core Modules & Mathematical Foundation

4.1 EWMA Latency Tracking & Circuit Breaker Logic

Standard load balancers use simple moving averages (SMA) which fail to react quickly to abrupt network degradation. The Smart Router implements an **Exponentially Weighted Moving Average (EWMA)** algorithm to weight recent latency metrics higher while smoothing random network jitter.

The calculated latency L_t at time t is defined by:

$$L_t = \alpha \cdot R_t + (1 - \alpha) \cdot L_{t-1}$$

Where R_t represents the raw ping response time of the current request, and $\alpha = 0.25$ is the smoothing decay factor. If a node fails three consecutive pings or its L_t exceeds a configurable threshold (e.g., 1500ms), the **Circuit Breaker** trips, changing the node state to **DEAD** and shifting 100% of traffic to healthy fallbacks.

4.2 Google Gemini AI Bytecode Intent Explainer

When an `eth_sendTransaction` or `eth_call` payload is intercepted, the router isolates the raw hex string (`data` field) and queries the Google Gemini API (`@google/genai`). The structured prompt instructs Gemini to:

1. Extract the 4-byte ERC-4337 or EVM method selector and match parameters against contract ABIs.
2. Translate high-risk calls (e.g., token transfers, proxy upgrades, unlimited approvals) into structured plain English.
3. Assign a real-time **Threat Rating** (`LOW` , `MEDIUM` , `CRITICAL`) based on identified risk indicators (e.g., unverified recipient, drainer signature patterns).

4.3 Zero-Cost Local State Dry-Run Sandbox

Before committing real gas fees, write transactions are executed in an isolated EVM simulation sandbox (`eth_call` with state overrides). The dry-run engine verifies:

- Exact gas consumption estimates versus user-provided gas limits.
- State diff outputs (token balance changes, allowance updates, owner shifts).
- Execution revert conditions, surfacing explicit error strings (e.g., `TRANSFER_FAILED` , `SlippageExceeded`) prior to mainnet broadcast.

5. Tech Stack & Software Architecture

Layer	Component	Technical Function & Details
Frontend Dashboard	HTML5, Vanilla JS, Tailwind CSS, Chart.js, Lucide Icons	Renders live telemetry graphs, interactive EWMA node latency curves, AI streaming logs, and raw JSON RPC consoles without heavy frameworks.
Backend Engine	Node.js (v18+), Express.js	High-throughput asynchronous JSON-RPC reverse proxy server, request parser, and node routing middleware.
AI Processing	Google Gemini API (<code>@google/genai</code>)	Generative AI engine performing zero-shot EVM bytecode intent decoding and threat classification.
Blockchain Middleware	Ethers.js / Web3 JSON-RPC	Handles RPC payload serialization, signature verification, gas estimation, and multi-provider state calls.

Environment Setup & Configuration Guide

1. Clone the repository and install core dependencies:

```
git clone https://github.com/your-username/web3-ai-smart-router.git
cd web3-ai-smart-router
npm install
```

2. Configure environment parameters in `.env` :

```
# Google Gemini AI Key
GEMINI_API_KEY=AiZaSyYourGoogleGeminiApiKeyHere

# Router Network Configuration
PORT=3000
DEFAULT_CHAIN_ID=1
RPC_CHECK_INTERVAL_MS=5000
EWMA_ALPHA=0.25
```

3. Launch the proxy server:

```
npm start
```

```
# Expected Console Verification:
# [SUCCESS] Smart Router backend listening on http://localhost:3000
# [AI ENGINE] Google Gemini API connection verified.
# [NODE MESH] Connected to 3 primary RPC endpoints. EWMA Engine active.
```

6. Live Demonstration & Judge Walkthrough

60-SECOND EVALUATION SEQUENCE

- 1. **Initialize Dashboard:** Navigate to `http://localhost:3000` to view real-time latency graphs and connected RPC nodes.
- 2. **Inspect Unreadable Payload:** In the Transaction Hub, select the `ERC-20 Transfer` raw hex template (`0xa9059cbb...`).
- 3. **Trigger AI Translation:** Click *Send Raw RPC Payload*. Observe the AI Terminal render a human-readable explanation: *"Transfer 500 USDC to 0x71C... with LOW threat rating."*
- 4. **Execute Dry-Run Simulation:** Click *Dry Run State Simulation* to preview gas usage and verify zero execution reverts without spending mainnet ETH.
- 5. **Demonstrate Fault Tolerance:** In the Node Matrix, click **Kill Endpoint** on Node 1. Watch the EWMA engine instantly shift traffic to Node 3 with zero dropped packets.
- 6. **MetaMask Integration:** Click *+ Add Network to MetaMask* and send a test transaction through the protected proxy port.

7. Comprehensive Technical Glossary

Term	Technical Definition & Context
RPC Node	A networked server hosting a blockchain client (e.g., Geth, Besu) that processes JSON-RPC requests for state reads and transaction execution.
EWMA Latency	Exponentially Weighted Moving Average metric that applies exponentially decreasing weights to older latency values, ensuring high sensitivity to recent network spikes.
MEV Shield	A privacy-preserving RPC routing mechanism that submits signed transaction bundles directly to private builder relays (e.g., Flashbots) rather than public mempools.
Circuit Breaker	An automated safety pattern that temporarily isolates degraded or non-responsive RPC nodes from the active pool to prevent cascading system failures.
State Dry-Run	EVM execution simulation against a read-only block state fork to evaluate contract execution logic, gas consumption, and potential revert reasons.
Bytecode Calldata	The raw hexadecimal payload appended to EVM transactions containing the 4-byte function signature and encoded parameters.